

Silver Layer — Cleaning & Enrichment

ServiceTrack Project | Execution Screenshots

Screenshot 1: Remove Duplicate Job Records



The screenshot shows a Databricks notebook interface with the title "Remove Duplicate Job Records". The notebook is written in Python and contains the following code:

```
1 # Drop duplicate records based on job_id column
2 df_jobs_dedup = df_jobs.dropDuplicates(["job_id"])
3
4 # Check record counts before and after deduplication to verify
5 print(f"Raw jobs count: {df_jobs.count()}")
6 print(f"Deducted jobs count: {df_jobs_dedup.count()}")
```

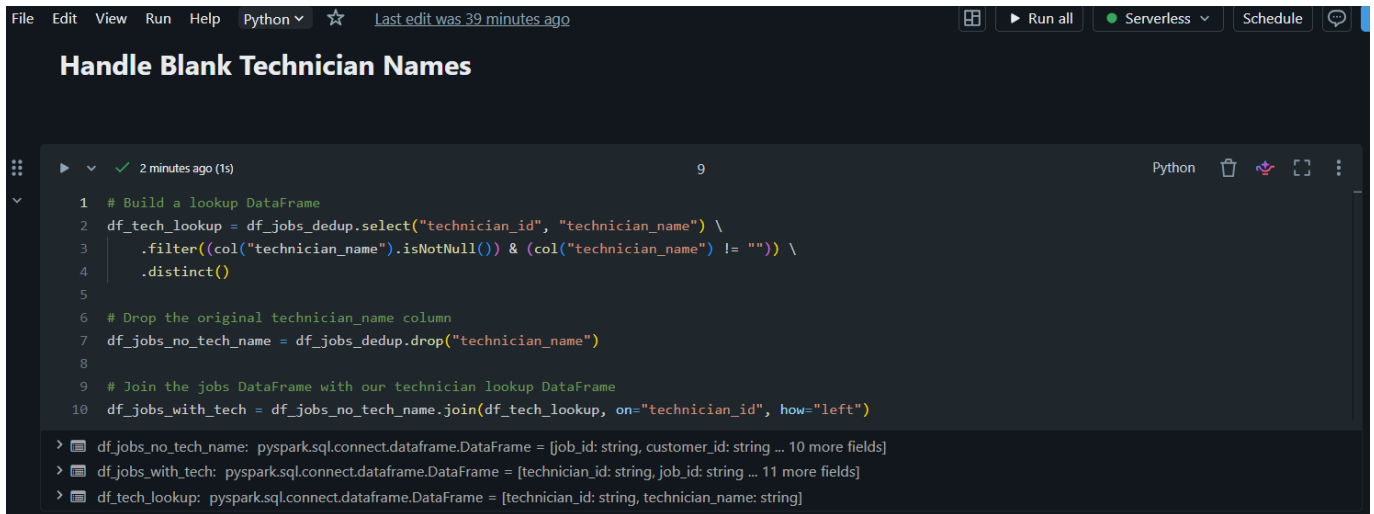
Below the code, the execution results are displayed:

```
> df_jobs_dedup: pyspark.sql.connect.dataframe.DataFrame = [job_id: string, customer_id: string ... 11 more fields]
Raw jobs count: 1510
Deducted jobs count: 1500
```

The interface also shows a status bar at the top with "File", "Edit", "View", "Run", "Help", "Python", and "Last edit was 36 minutes ago". The bottom right corner indicates "Checked for improvements".

Duplicate `job_id` rows removed using `dropDuplicates()` — row count reduced from 1510 to 1500.

Screenshot 2: Handle Blank Technician Names



The screenshot shows a code editor interface with a dark theme. At the top, there's a menu bar with 'File', 'Edit', 'View', 'Run', 'Help', and a 'Python' dropdown. To the right of the menu bar, there's a star icon, a link 'Last edit was 39 minutes ago', and buttons for 'Run all', 'Serverless', 'Schedule', and a chat icon. Below the menu bar, the title 'Handle Blank Technician Names' is displayed. The main area contains a code editor with a file explorer on the left showing a folder '2 minutes ago (1s)' containing a file '9'. The code is in Python and uses PySpark. It defines a lookup DataFrame from 'df_jobs_dedup', filters for non-null and non-empty technician names, and then joins this lookup back to the original DataFrame to fill in blank names. Below the code editor, there's a console output showing the schema of the DataFrames: 'df_jobs_no_tech_name' has 10 fields, 'df_jobs_with_tech' has 11 fields, and 'df_tech_lookup' has 2 fields.

```
1 # Build a lookup DataFrame
2 df_tech_lookup = df_jobs_dedup.select("technician_id", "technician_name") \
3     .filter((col("technician_name").isNotNull()) & (col("technician_name") != "")) \
4     .distinct()
5
6 # Drop the original technician_name column
7 df_jobs_no_tech_name = df_jobs_dedup.drop("technician_name")
8
9 # Join the jobs DataFrame with our technician lookup DataFrame
10 df_jobs_with_tech = df_jobs_no_tech_name.join(df_tech_lookup, on="technician_id", how="left")

> df_jobs_no_tech_name: pyspark.sql.connect.dataframe.DataFrame = [job_id: string, customer_id: string ... 10 more fields]
> df_jobs_with_tech: pyspark.sql.connect.dataframe.DataFrame = [technician_id: string, job_id: string ... 11 more fields]
> df_tech_lookup: pyspark.sql.connect.dataframe.DataFrame = [technician_id: string, technician_name: string]
```

A technician lookup table is built and joined back to fill in blank `technician_name` values using `technician_id`.

Screenshot 3: Create job_status_flag Column

```
1 # Create job_status_flag: Delayed / On Time / Not Completed
2 df_jobs_duration = df_jobs_duration.withColumn(
3     "job_status_flag",
4     when(col("completed_date").isNull(), "Not Completed")
5     .when(col("completed_date") > col("promised_date"), "Delayed")
6     .otherwise("On Time")
7 )
8
9 # Quick check: see the distribution of the new flag
10 display(df_jobs_duration.groupBy("job_status_flag").count())
```

> [See performance \(1\)](#) ✓ Checked for improvements

> df_jobs_duration: pyspark.sql.connect.dataframe.DataFrame = [technician_id: string, job_id: string ... 13 more fields]

	job_status_flag	count
1	On Time	833
2	Delayed	297
3	Not Completed	370

↓ 3 rows | 1.817s runtime Refreshed 4 minutes ago

New job_status_flag column added and validated — 833 On Time, 297 Delayed, and 370 Not Completed jobs.

Screenshot 4: Join Datasets to Create Enriched Fact Table

FileEditViewRunHelpPython ☆Last edit was 1 minute ago

Join Datasets to Create Enriched Fact Table

Run allServerlessSchedule

07:23 PM (5s)23Python

```
1 # Join jobs with customers
2 df_joined_cust = df_jobs_cleaned.join(df_customers_cleaned, on="customer_id", how="left")
3
4 # Join the result with devices
5 df_enriched = df_joined_cust.join(df_devices_cleaned, on="device_id", how="left")
6
7 display(df_enriched.limit(5))
```

[See performance \(1\)](#) ✓ Checked for improvements

df_enriched: pyspark.sql.connect.dataframe.DataFrame = [device_id: string, customer_id: string ... 23 more fields]

df_joined_cust: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, technician_id: string ... 18 more fields]

Table +

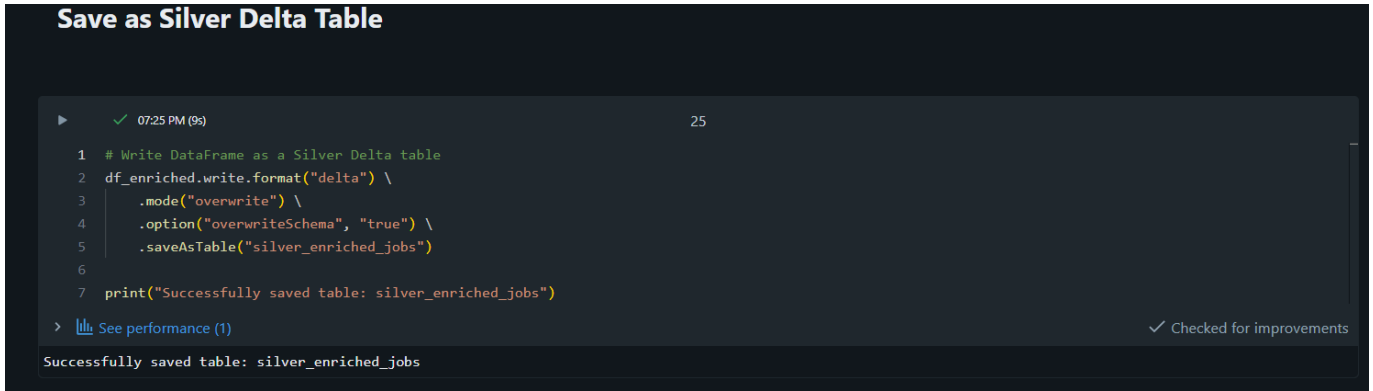
	device_id	customer_id	technician_id	job_id	issue_type	job_status	received_date	promised_date	
1	DEV019	CUST0295	T005	JOB01181	Battery Issue	Completed	2024-01-29	2024-02-03	2024
2	DEV035	CUST0108	T006	JOB00070	Keyboard Fault	Completed	2024-02-29	2024-03-05	2024
3	DEV020	CUST0001	T007	JOB00991	Software Crash	Cancelled	2024-03-13	2024-03-18	2024
4	DEV022	CUST0273	T008	JOB01238	Hard Disk Failure	Pending	2024-01-23	2024-01-28	null
5	DEV022	CUST0452	T005	JOB00056	Screen Issue	Completed	2024-04-20	2024-03-03	2024

5 rows | 5.002s runtime

Refreshed 43 minutes ago

Cleaned service jobs joined with customers and devices to form a single enriched fact table.

Screenshot 5: Save as Silver Delta Table



The screenshot shows a Databricks notebook interface with a dark theme. The notebook title is "Save as Silver Delta Table". The code editor displays a Python script with 7 lines. Line 1 is a comment: "# Write DataFrame as a Silver Delta table". Lines 2-5 use the `df_enriched.write.format("delta")` method chain to write the `df_enriched` DataFrame to a Delta table named `silver_enriched_jobs` in `overwrite` mode, with `overwriteSchema` set to `true`. Line 6 is an empty line, and line 7 prints a success message: `print("Successfully saved table: silver_enriched_jobs")`. Below the code editor, a status bar shows a green checkmark, the time "07:25 PM (9s)", and the page number "25". A link "See performance (1)" is visible. At the bottom, a message box states "Successfully saved table: silver_enriched_jobs".

```
1 # Write DataFrame as a Silver Delta table
2 df_enriched.write.format("delta") \
3     .mode("overwrite") \
4     .option("overwriteSchema", "true") \
5     .saveAsTable("silver_enriched_jobs")
6
7 print("Successfully saved table: silver_enriched_jobs")
```

> [See performance \(1\)](#) ✓ Checked for improvements

Successfully saved table: silver_enriched_jobs

Final enriched DataFrame saved as the `silver_enriched_jobs` Delta table.